Tema 3

“Búsqueda Heurística”

Año Académico 2026/27

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

Tema 3: Búsqueda heurística

Tema 4: Búsqueda heurística avanzada

Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

Tema 3: Búsqueda heurística

- Funciones heurísticas
- Algoritmo A^*
- Completitud y Optimalidad

Tema 4: Búsqueda heurística avanzada

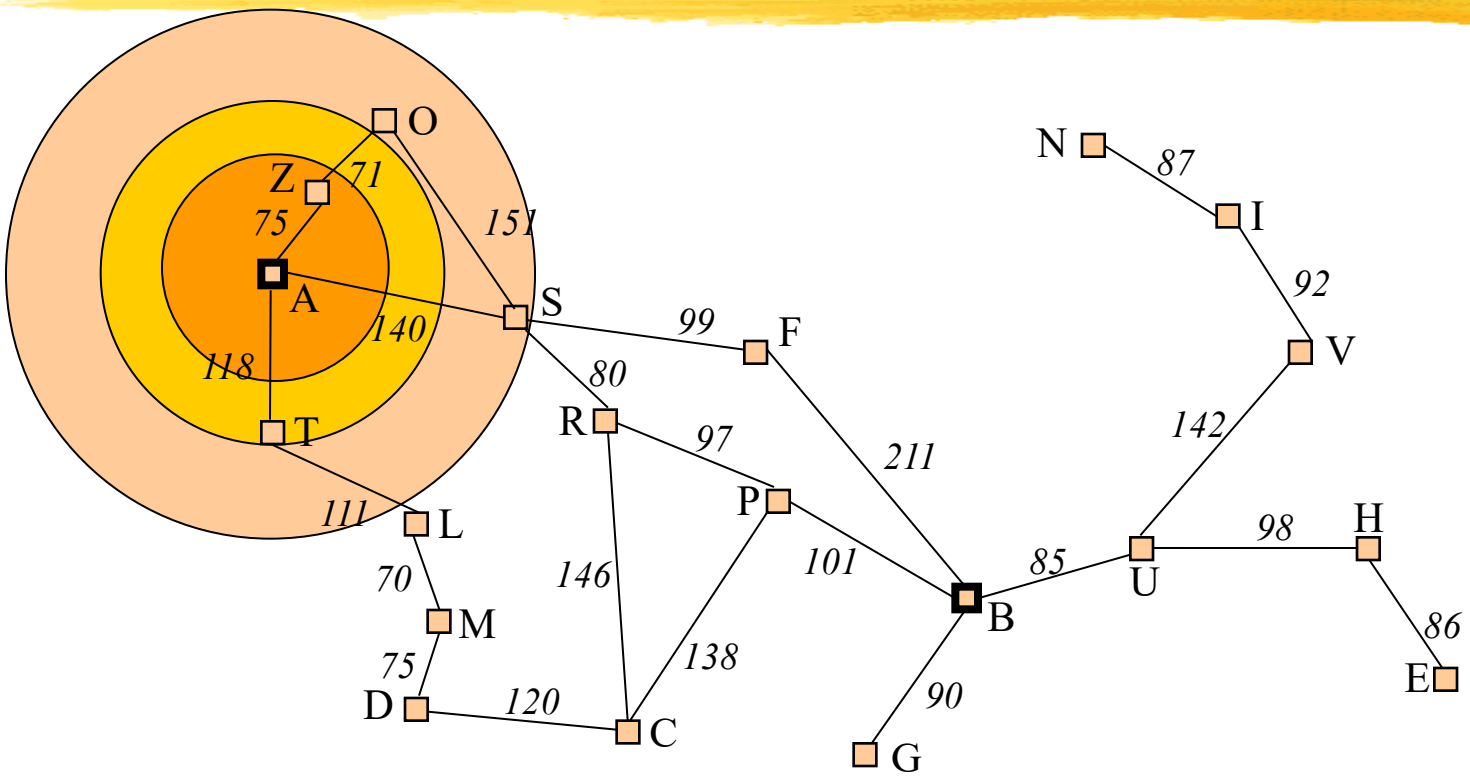
Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

Tema 2: Búsqueda de coste uniforme

- La búsqueda de coste uniforme enumera sucesivamente todos los nodos del espacio de estados por costes (valores de g) *crecientes*
- La búsqueda de coste uniforme es *completa* y *óptima* (el argumento se basa en que los valores de g crecen a lo largo de todos los caminos del árbol de búsqueda).
- La búsqueda de coste uniforme degenera en la búsqueda en amplitud si $g(n) = profundidad(n)$ para todos los nodos n
- Se heredan todas sus propiedades "negativas" de complejidad en el peor caso. Si $\hat{g}$ es el coste del camino al mejor nodo meta, y ε el coste mínimo de los operadores, entonces:
  - Complejidad en *tiempo* en el peor caso: $f_w \in O(b^{\hat{g}/\varepsilon})$
  - Complejidad en *espacio* en el peor caso: $f_w \in O(b^{\hat{g}/\varepsilon})$

Tema 2: Lógica de la búsqueda de coste uniforme



$g = 80$ $g = 120$ $g = 160$

Heurísticas

Heurística (griego: heuriskein): “encontrar”, “descubrir”

Inteligencia Artificial:

- compila conocimiento “empírico” sobre un problema / un entorno

Interpretación “fuerte”:

- una heurística *suele facilitar* la resolución de un problema, pero *no garantiza* que se resuelva
- una heurística es una “regla de tres” para un problema
- búsqueda: optimalidad o incluso completitud no garantizados

Interpretación “débil”:

- método riguroso + información heurística
- información heurística puede mejorar el *rendimiento medio* de un método de resolución de problemas, pero no garantiza una mejora en el *peor caso*
- búsqueda: mejora de complejidad no garantizado

Funciones heurísticas

Funciones heurísticas para búsqueda en el espacio de estados:

- estiman de adecuación de un nodo para ser expandido
- métodos de búsqueda "*el mejor primero*" eligen el nodo más prometedor para expandir

Heurística usual: "distancia" hacia la meta

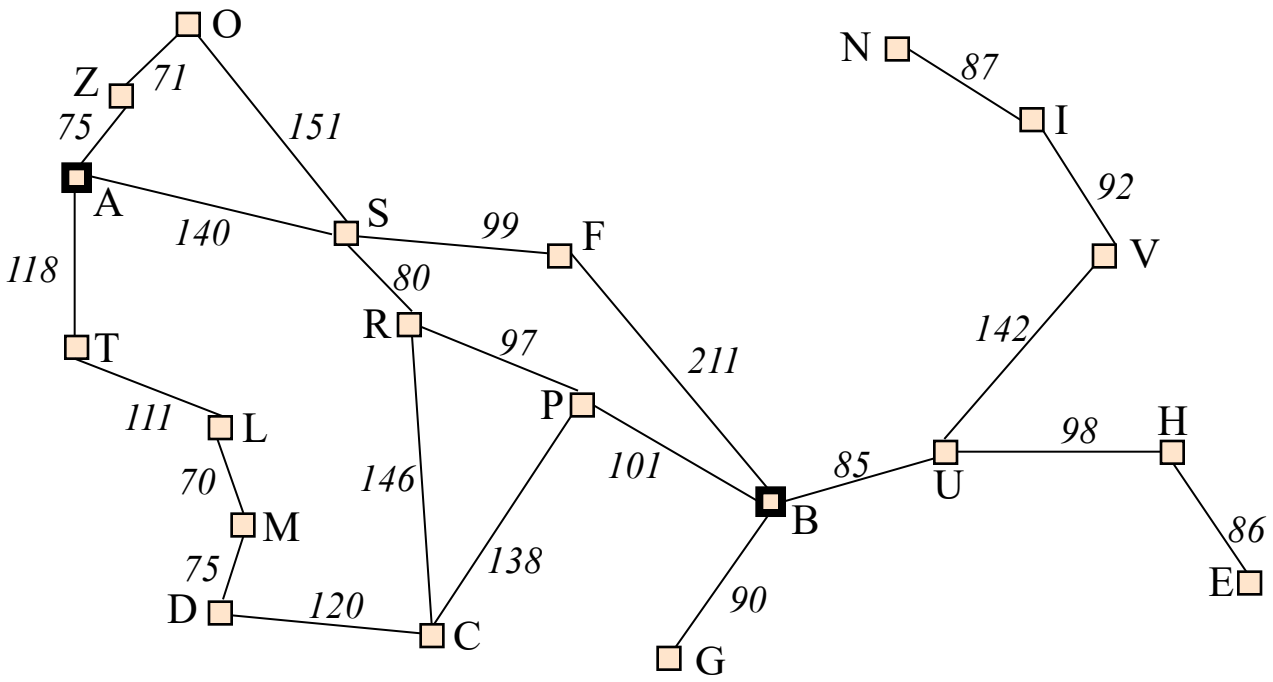
- $h : N \rightarrow \aleph$ mide el coste *real* desde el nodo n hasta el nodo meta más cercano
- $h^* : N \rightarrow \aleph$ es una ***función heurística*** que estima el valor de $h(n)$
- una función heurística h^* es ***optimista***, si $h^*(n) \leq h(n)$ para todo nodo n

  (En algunas fuentes estas funciones se denominan "**admisibles".**)

Ejemplos de funciones heurísticas optimistas:

- mundo de los bloques: número de bloques *descolocados*
- encontrar rutas: distancia en *línea recta* hasta un nodo meta

Función heurística para encontrar rutas



	h^*
A	366
B	0
C	160
D	242
E	161
F	178
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	98
R	193
S	253
T	329
U	80
V	199
Z	374

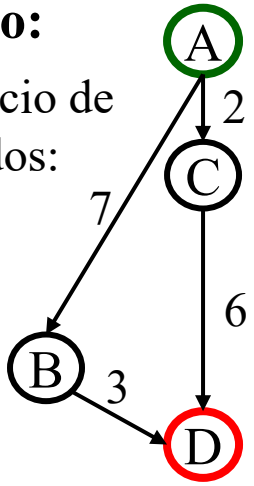
Aproximación naíf

Búsqueda avara:

- entre las hojas del árbol de búsqueda, elegir el nodo de valor h^* mínimo
- e.d., el nodo hoja del que pensamos que más cerca esté de la meta

Ejemplo:

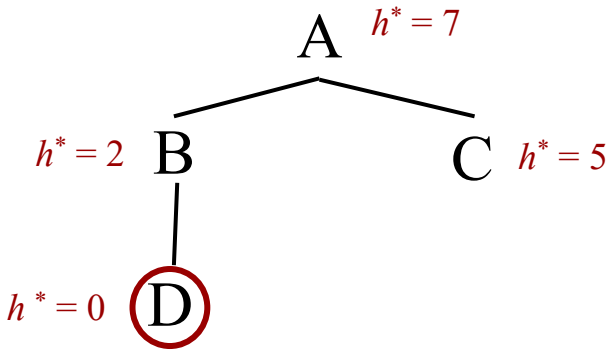
Espacio de estados:



Función heurística:

	h^*
A	7
B	2
C	5
D	0

Árbol de búsqueda:



Problema:

- La búsqueda avara **no** es óptima…
- No solo hay que tomar en cuenta cómo de cerca está un nodo de la meta, sino también cuánto ha costado llegar hasta él desde el estado inicial

Búsqueda A*

Idea:

- minimizar el coste estimado *total* de un camino en el árbol de búsqueda
- combinar
  - el coste para llegar al nodo n (se conoce exactamente: g), y
  - el coste aproximado para llegar a un nodo meta desde el nodo n (estimado por la función heurística h^*)

Función heurística de A*:

- $f(n) = g(n)+h(n)$: coste *real* del plan (camino) de mínimo coste que pasa por n
- $f^*(n) = g(n)+h^*(n)$: *estimación* de f

Estrategia A* :

- entre las hojas del árbol de búsqueda, elegir el nodo de valor f^* mínimo

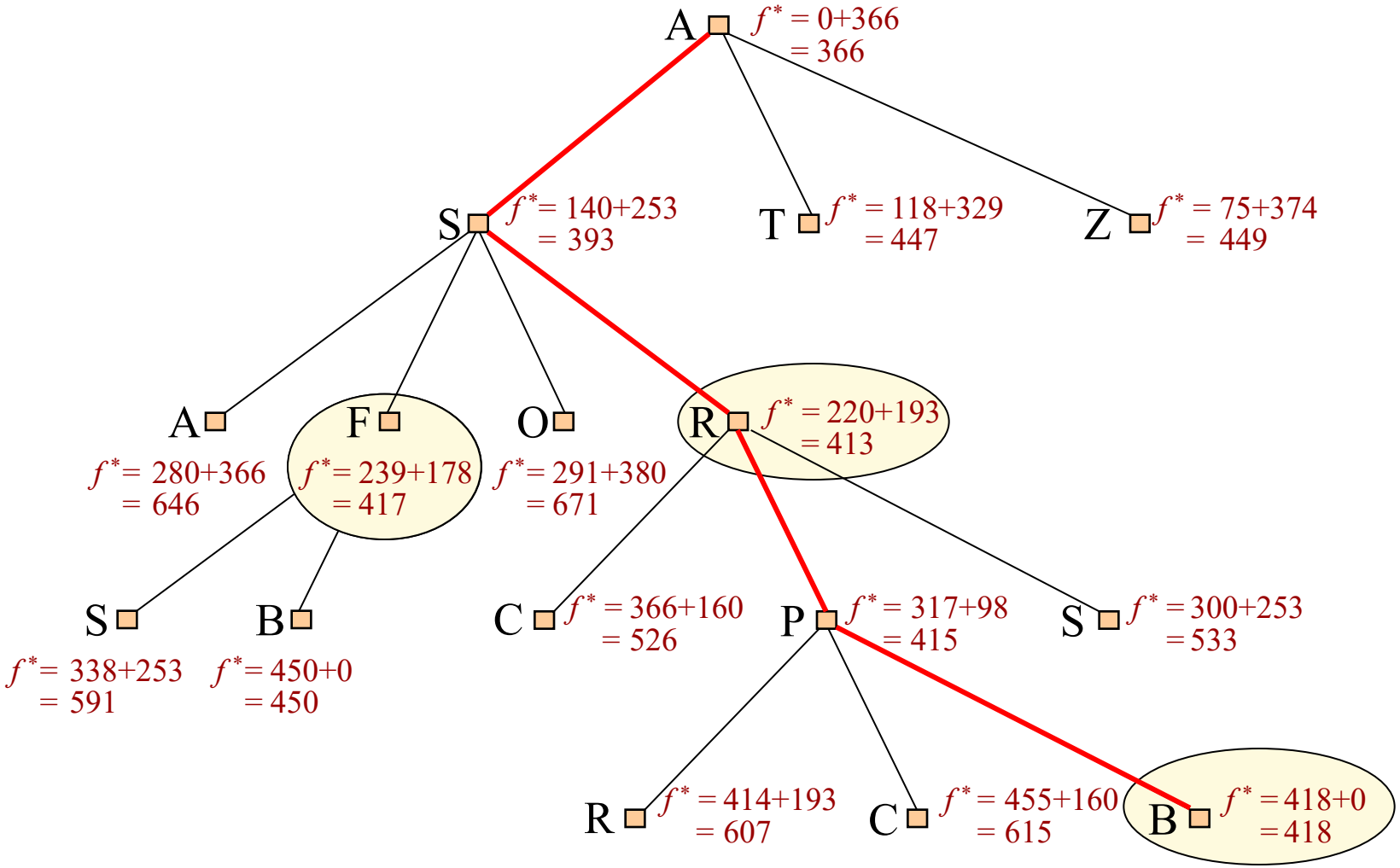
El Algoritmo A*

Algoritmo A* :

- se basa en la búsqueda general
- almacenar el valor *g* de cada nodo expandido
- mantener la lista *abierta* ordenada por valores crecientes de f^*
- insertar nuevos nodos en *abierta* según sus valores f^*

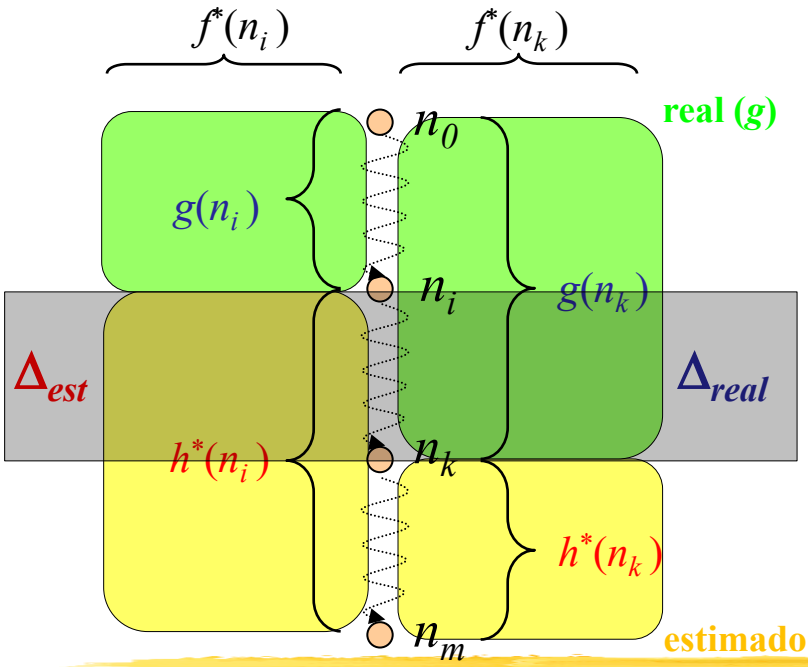
{A*}
abierta ← s_0
Repetir
  Si *vacío?*(abierta) **entonces**
    devolver(negativo)
  nodo ← primero(abierta)
  Si *meta?*(nodo) **entonces**
    devolver(nodo)
  sucesores ← *expandir*(nodo)
  Para cada n∈sucesores **hacer**
    n.padre ← nodo
    ordInsertar(n,abierta, f^*)
  Fin {repetir}

Ejemplo: Búsqueda A*



Búsqueda A*: valores de f^*

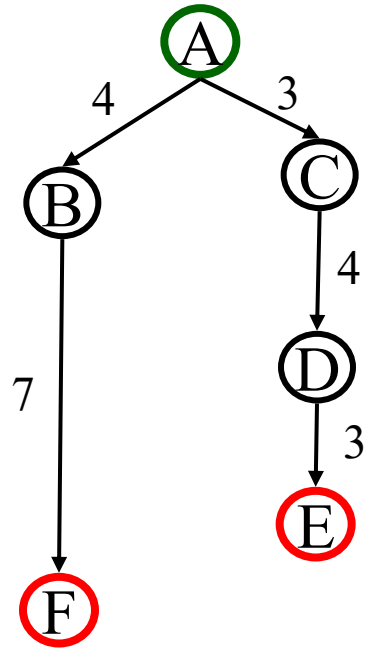
- En la búsqueda de coste uniforme, los valores de g crecen a lo largo de todos los caminos del árbol de búsqueda, porque en cada paso se suma el coste de un operador (que es un número natural positivo)
- En el ejemplo de la transparencia anterior, los valores de f^* también crecen a lo largo de todos los caminos del árbol de búsqueda
- Pero esto no siempre ha de ser así:



- En un camino del árbol de búsqueda, sea n_k un sucesor (no necesariamente directo) de n_i
- entonces el valor de $f^*(n_i)$ contiene una estimación del coste de llegar de n_i a n_k (Δ_{est}), y $f^*(n_k)$ el coste real (Δ_{real})
- Es decir, se puede obtener el valor de $f^*(n_k)$ a partir de $f^*(n_i)$ sumando Δ_{real} y restando Δ_{est}
- Es posible que $f^*(n_k) < f^*(n_i)$ si $\Delta_{est} > \Delta_{real}$

Ejemplo

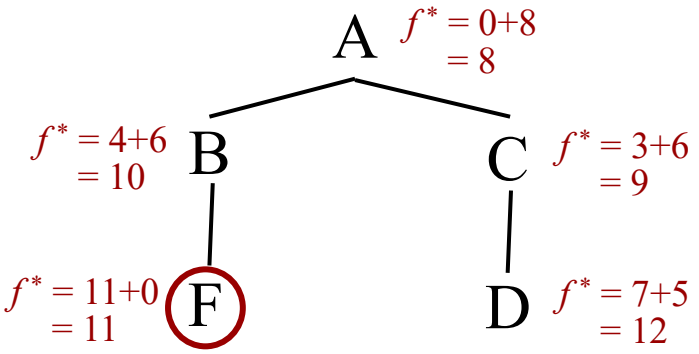
Espacio de estados:



	h^*
A	8
B	6
C	6
D	5
E	0
F	0

- Estado inicial: A
- Estados meta: E, F
- $g(E)=10 < 11=g(F)$

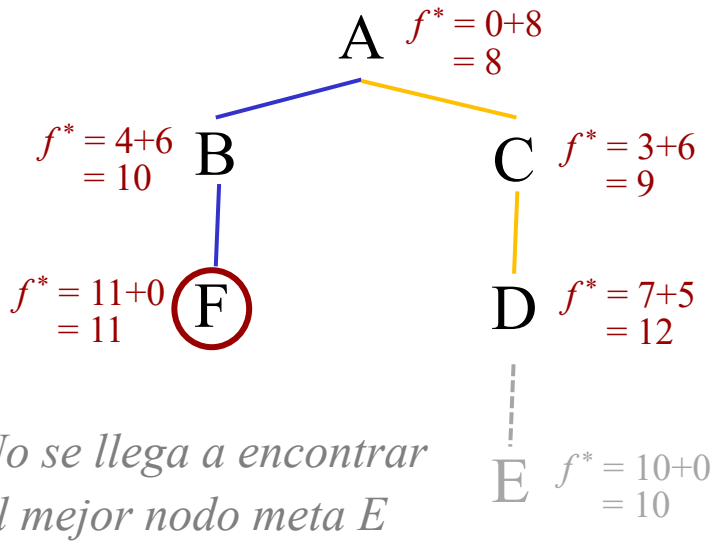
Árbol de búsqueda:



- A^* no encuentra el mejor nodo meta E, sino F
- h^* no es optimista: $h^*(D) = 5 > 3 = h(D)$

Ejemplo (continuado)

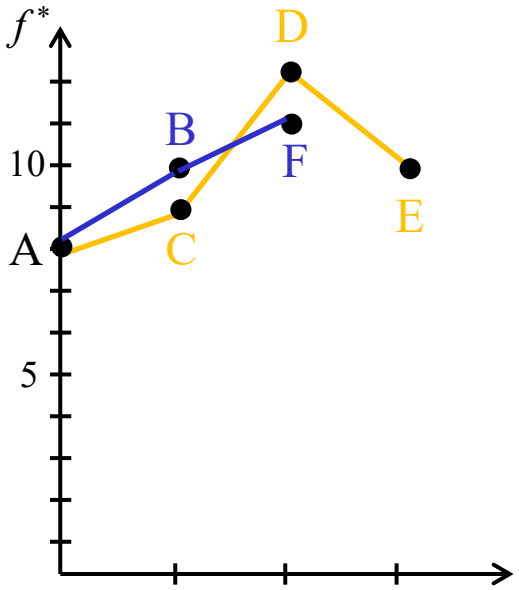
- Árbol de búsqueda:



No se llega a encontrar el mejor nodo meta E

- En el camino de A al mejor nodo meta E ($f^*(E) = 10$) hay un "pico" D ($f^*(D) = 12$) que la búsqueda A^* no ha podido superar

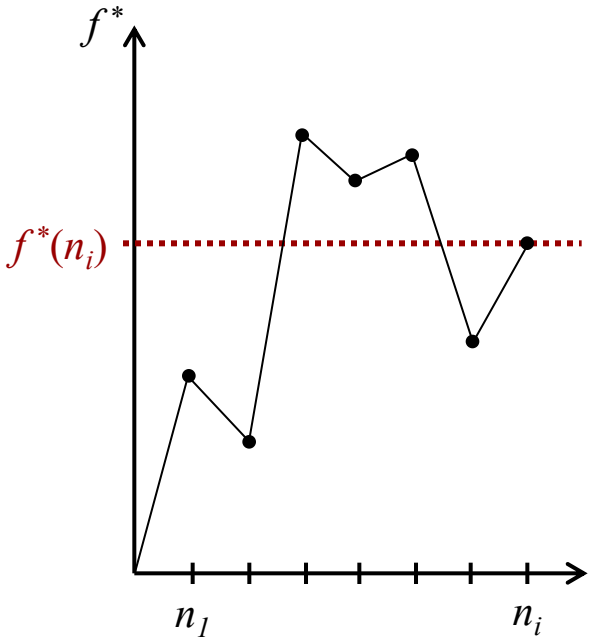
- Variación de los valores de f^* a lo largo de los caminos del árbol de búsqueda



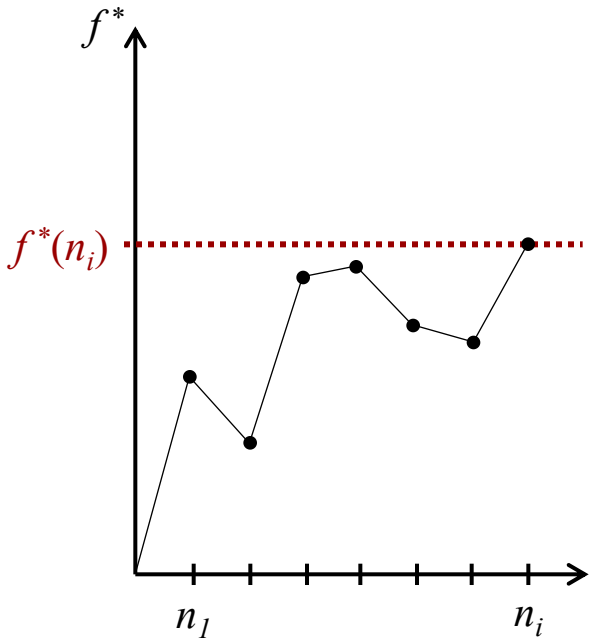
Valores de f^* en árboles de búsqueda A*

Tipos de variación de los valores de f^* a lo largo de un camino desde la raíz hasta un nodo n_i

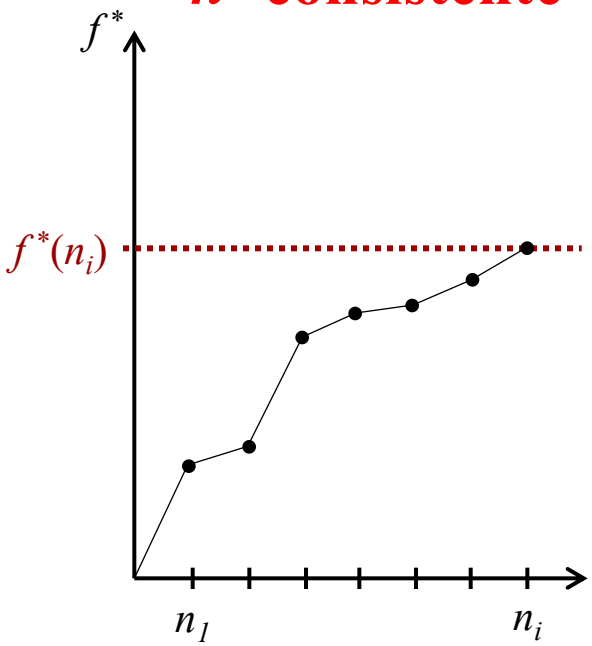
h^* consistente



(a) variable



(b) acotado por $f^*(n_i)$



(c) monótono creciente

Funciones heurísticas consistentes

Definición:

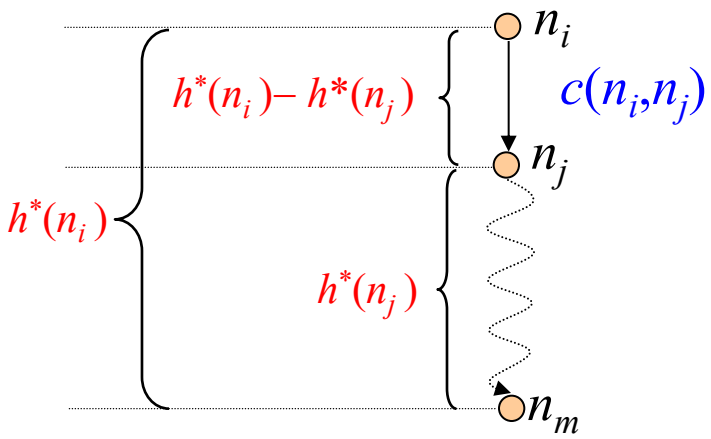
Si para todo nodo n_i y todo sucesor directo n_j de n_i se cumple que

$$h^*(n_i) - h^*(n_j) \leq c(n_i,n_j)$$

entonces h^* es *consistente*

Interpretación intuitiva:

- h^* estima también *implícitamente* el coste de cada operador
- h^* es consistente si *subestima* el coste de todos los operadores



Nótese:

- Si h^* es *consistente*, entonces también es *optimista*
- Pero existen funciones heurísticas h^* *optimistas* que **no** son *consistentes*

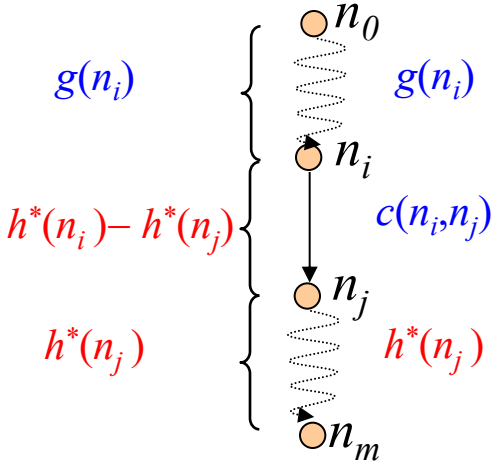
Monotonía de f^* con función heurística consistente

Lema 1: Si h^* es *consistente*, entonces f^* crece de forma *débilmente monótona* en todos los caminos del árbol de búsqueda, es decir: si n_j es sucesor directo de n_i, entonces

$$f^*(n_i) \leq f^*(n_j)$$

Prueba:

$$\begin{aligned} h^*(n_i) - h^*(n_j) &\leq c(n_i,n_j) \\ g(n_i) + h^*(n_i) - h^*(n_j) &\leq g(n_i) + c(n_i,n_j) \quad (+ g(n_i)) \\ g(n_i) + h^*(n_i) - h^*(n_j) &\leq g(n_j) \quad (\text{Árbol de búsqueda}) \\ g(n_i) + h^*(n_i) &\leq g(n_j) + h^*(n_j) \quad (+ h^*(n_j)) \\ f^*(n_i) &\leq f^*(n_j) \quad (\text{Def. } f^*) \end{aligned}$$

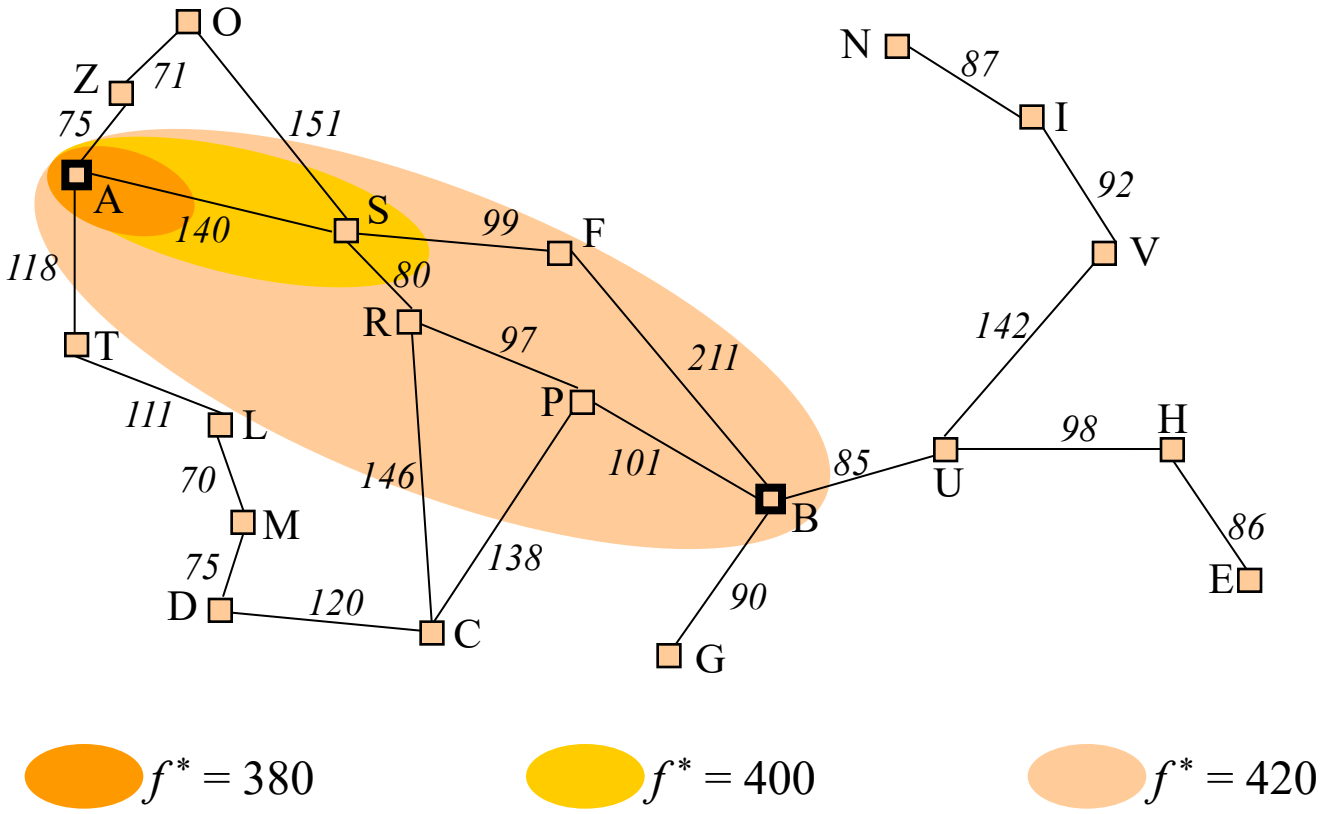


Corolario 1: Sea n_m el mejor nodo meta. Si h^* es *consistente*, entonces el algoritmo A^* expande todos los nodos n_i tal que

$$f^*(n_i) \leq f^*(n_m)$$

La estrategia de A* con función heurística consistente

- Si nos *alejamos* de la meta: g crece y h^* crece, por lo que f^* crece "mucho"
- Si nos *acercamos* de la meta: g crece y h^* disminuye, por lo que f^* crece "poco"



Cota de f^* con función heurística optimista

Lema 2: Sea *camino_a*(n_m) el conjunto de nodos en el camino desde la raíz a un nodo meta n_m cualquiera. Si h^* es optimista, entonces para todos los nodos $n_j \in$ *camino_a* (n_m) se verifica que

$$f^*(n_j) \leq f^*(n_m)$$

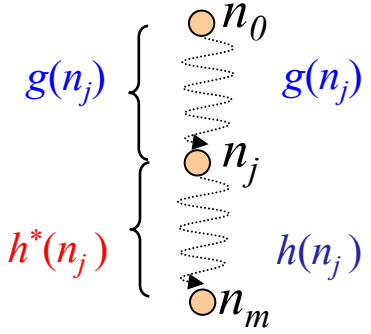
Prueba:

$$\begin{aligned} h^*(n_j) &\leq h(n_j) \\ g(n_j) + h^*(n_j) &\leq g(n_j) + h(n_j) \\ g(n_j) + h^*(n_j) &\leq g(n_m) \\ f^*(n_j) &\leq f^*(n_m) \end{aligned}$$

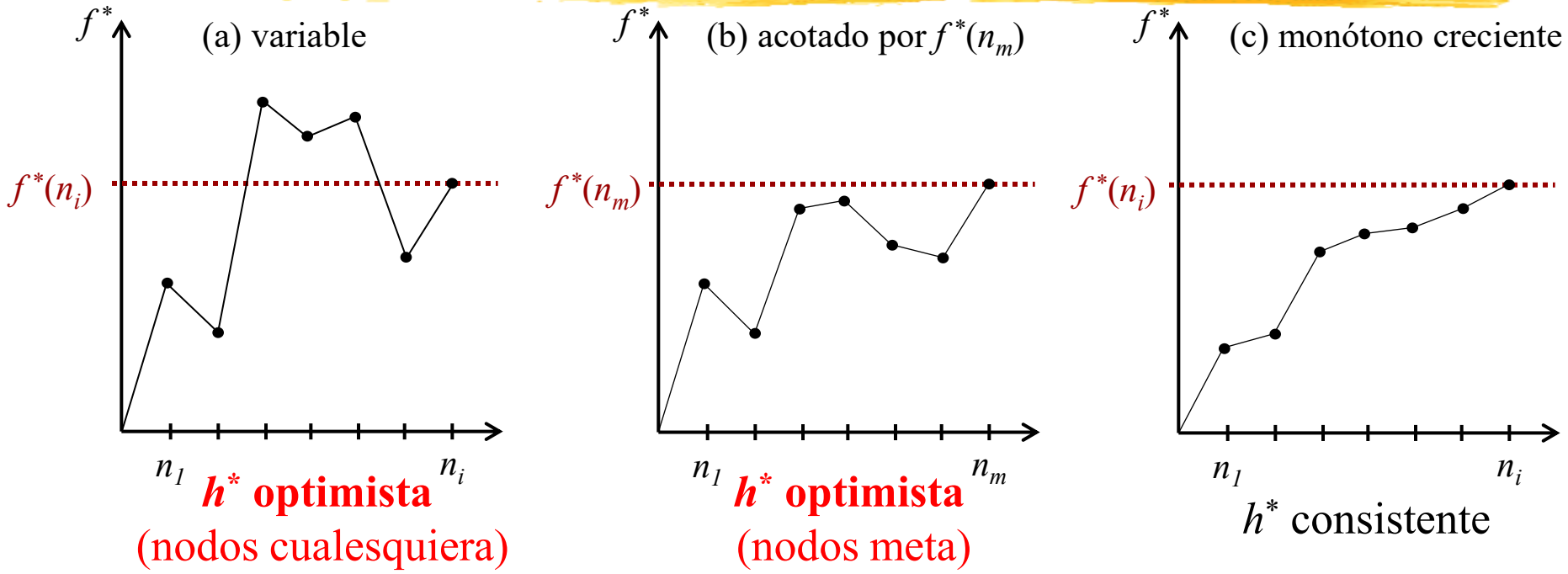
$+ g(n_j)$

Árbol de búsqueda

Def. f^* h^* *optimista*



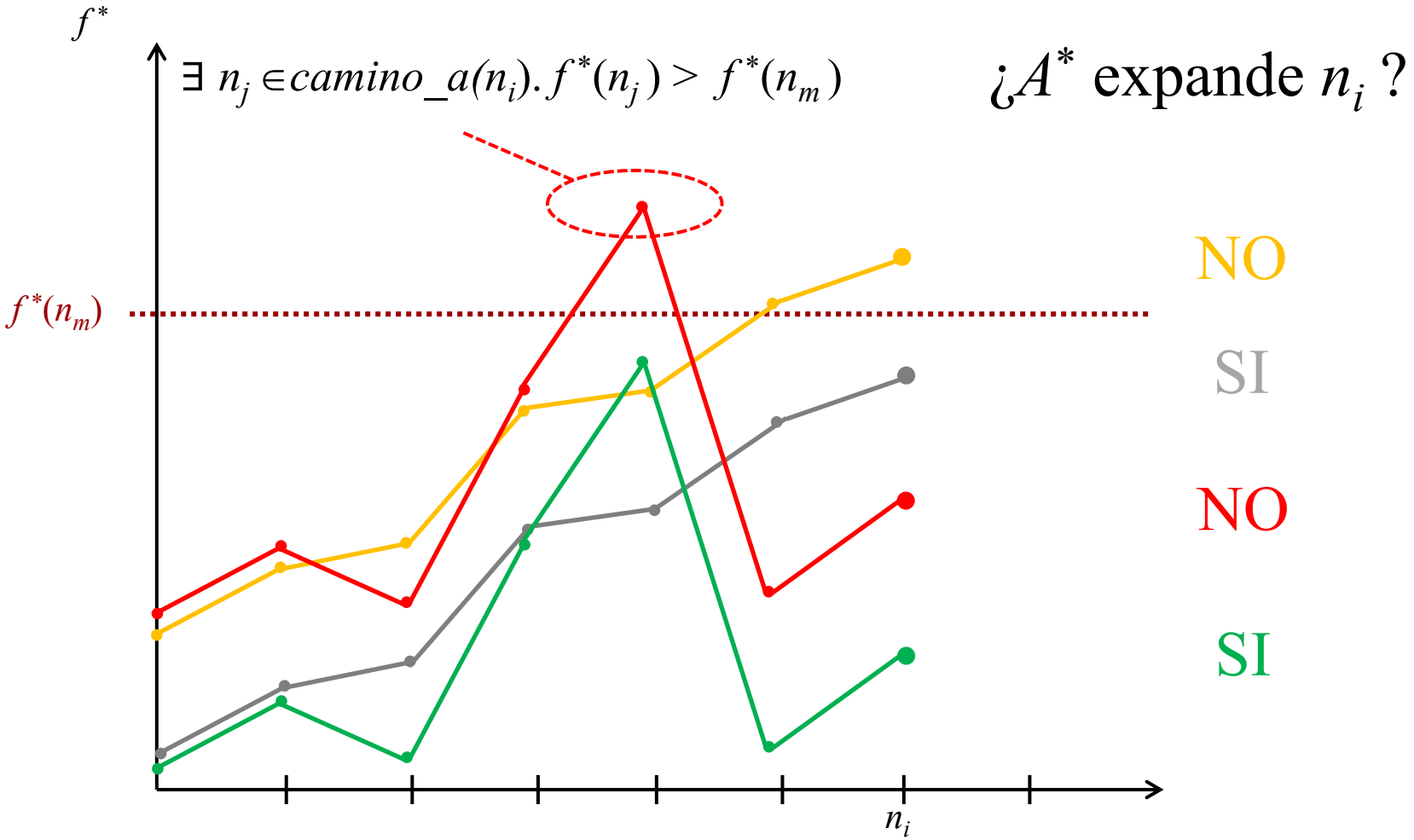
Valores de f^* en árboles de búsqueda A^*



Corolario 2: Sea n_m el mejor nodo meta, y *camino_a*(n_i) el conjunto de nodos n_j en el camino desde la raíz a un nodo n_i cualquiera (n_i incluido). Si h^* es *optimista*, entonces el algoritmo A^* expande todos los nodos n_i tal que

$$\forall n_j \in camino_a\,(n_i).\, f^*(n_j) \leq f^*(n_m)$$

Ejemplo Corolario 2



Optimalidad de A^*

Teorema 1: Si h^* es optimista (admisible), entonces el método A^* es *óptimo*

Prueba:
Sean n_{m1} y n_{m2} dos nodos meta cualesquiera distintos, y n_{m1} el nodo meta de menor coste (es decir: $g(n_{m1})<g(n_{m2})$).

1. $f^*(n_{m1}) = g(n_{m1})+h^*(n_{m1}) = g(n_{m1})+0$ y
   $f^*(n_{m2}) = g(n_{m2})+h^*(n_{m2}) = g(n_{m2})+0$
   Con la suposición se sigue que $f^*(n_{m1}) < f^*(n_{m2})$
   (por las definiciones de f^* y g y dado que h^* es optimista)
2. Supongamos que n_{m2} se encuentra antes que n_{m1}. Puesto que el algoritmo A^* expande los nodos del árbol por valores crecientes de f^*, debe existir un nodo $n_i \in camino_a(n_{m1})$ tal que $f^*(n_{m2}) \leq f^*(n_i)$.
3. Pero por el lema 2 y (1) sabemos que $\forall\ n_j \in camino_a(n_{m1})$. $f^*(n_j) \leq f^*(n_{m1}) < f^*(n_{m2})$. Contradicción (no puede existir el nodo n_i de (2))
4. Por tanto, para cualesquiera dos nodos meta, A^* encuentra el de menor coste antes, lo que implica que es óptimo.

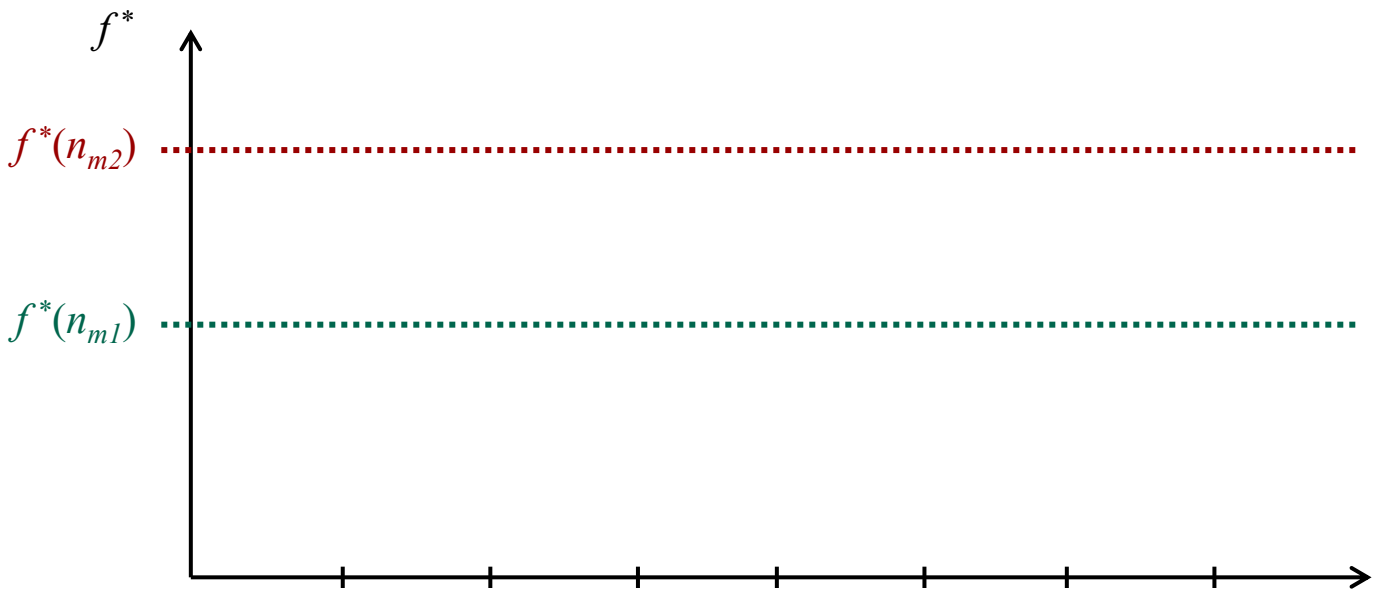
Optimalidad de A*

Visualización de la demostración

Sean n_{m1} y n_{m2} dos nodos meta cualesquiera distintos, y n_{m1} el nodo meta de menor coste (es decir: $g(n_{m1})<g(n_{m2})$).

1. $f^*(n_{m1}) = g(n_{m1})+h^*(n_{m1}) = g(n_{m1})+0$ y
   $f^*(n_{m2}) = g(n_{m2})+h^*(n_{m2}) = g(n_{m2})+0$
   Con la suposición se sigue que $f^*(n_{m1}) < f^*(n_{m2})$

   (por las definiciones de f^* y g y dado que h^* es optimista)

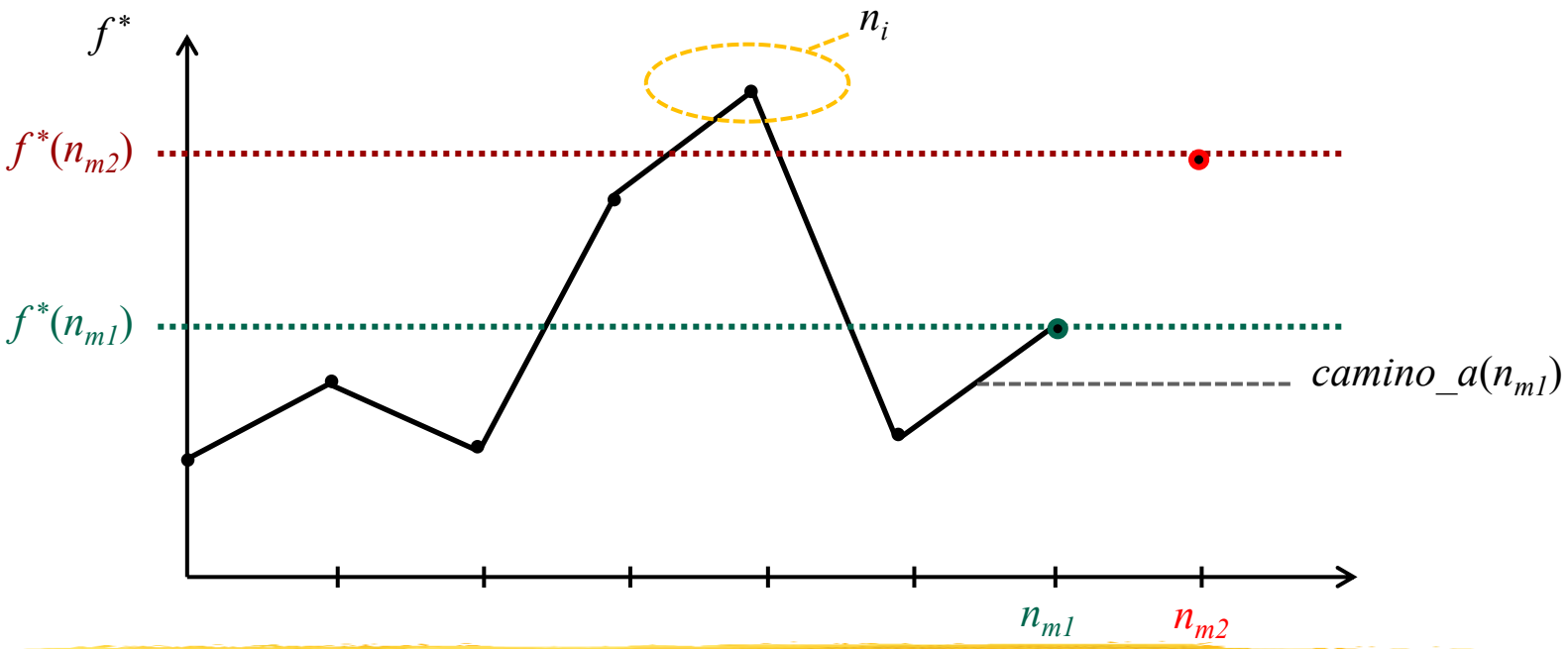


Optimalidad de A^*

Visualización de la demostración

Sean n_{m1} y n_{m2} dos nodos meta cualesquiera distintos, y n_{m1} el nodo meta de menor coste (es decir: $g(n_{m1})<g(n_{m2})$).

2. Supongamos que n_{m2} se encuentra antes que n_{m1}. Puesto que el algoritmo A^* expande los nodos del árbol por valores crecientes de f^*, debe existir un nodo $n_i \in camino_a(n_{m1})$ tal que $f^*(n_{m2}) \leq f^*(n_i)$.

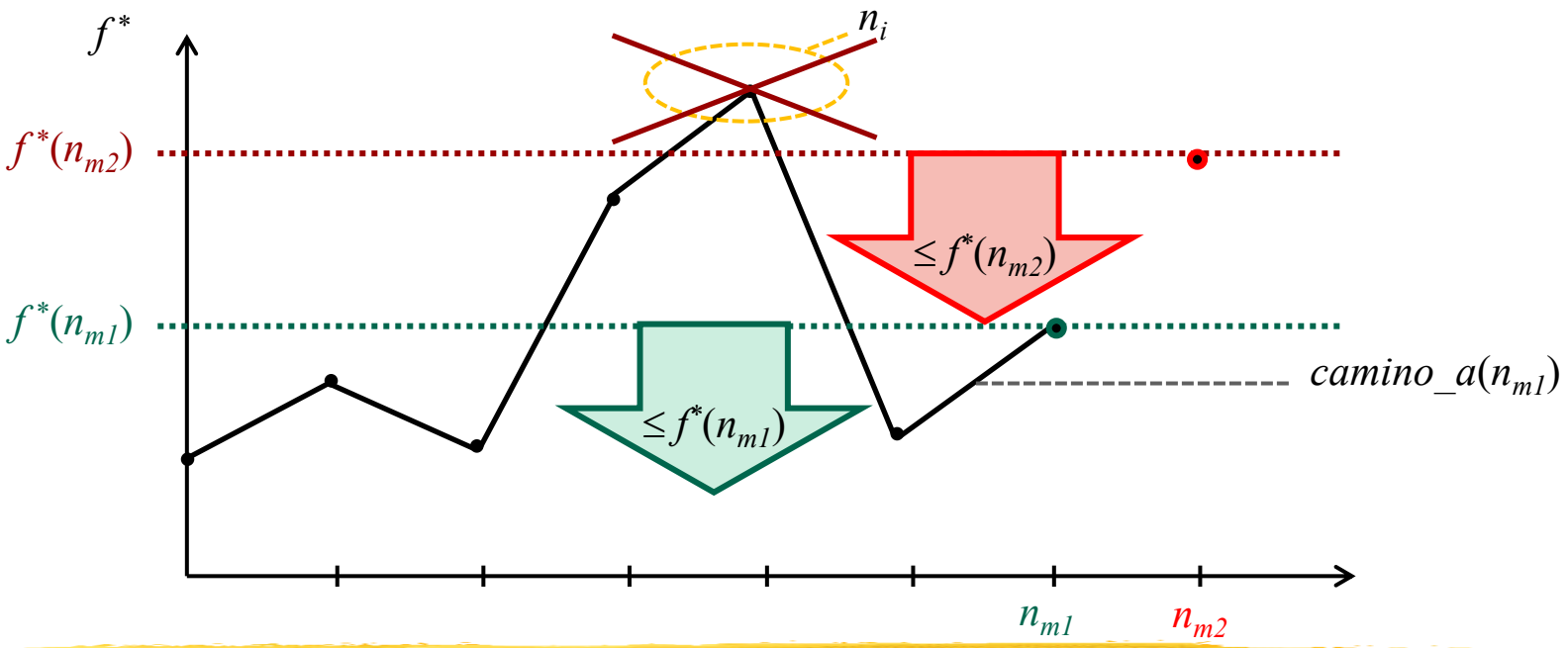


Optimalidad de A*

Visualización de la demostración

Sean n_{m1} y n_{m2} dos nodos meta cualesquiera distintos, y n_{m1} el nodo meta de menor coste (es decir: $g(n_{m1})<g(n_{m2})$).

3. Pero por el lema 2 y (1) sabemos que $\forall\ n_j \in camino_a(n_{m1}).\ f^*(n_j) \leq f^*(n_{m1}) < f^*(n_{m2})$. Contradicción (no puede existir el nodo n_i de (2))

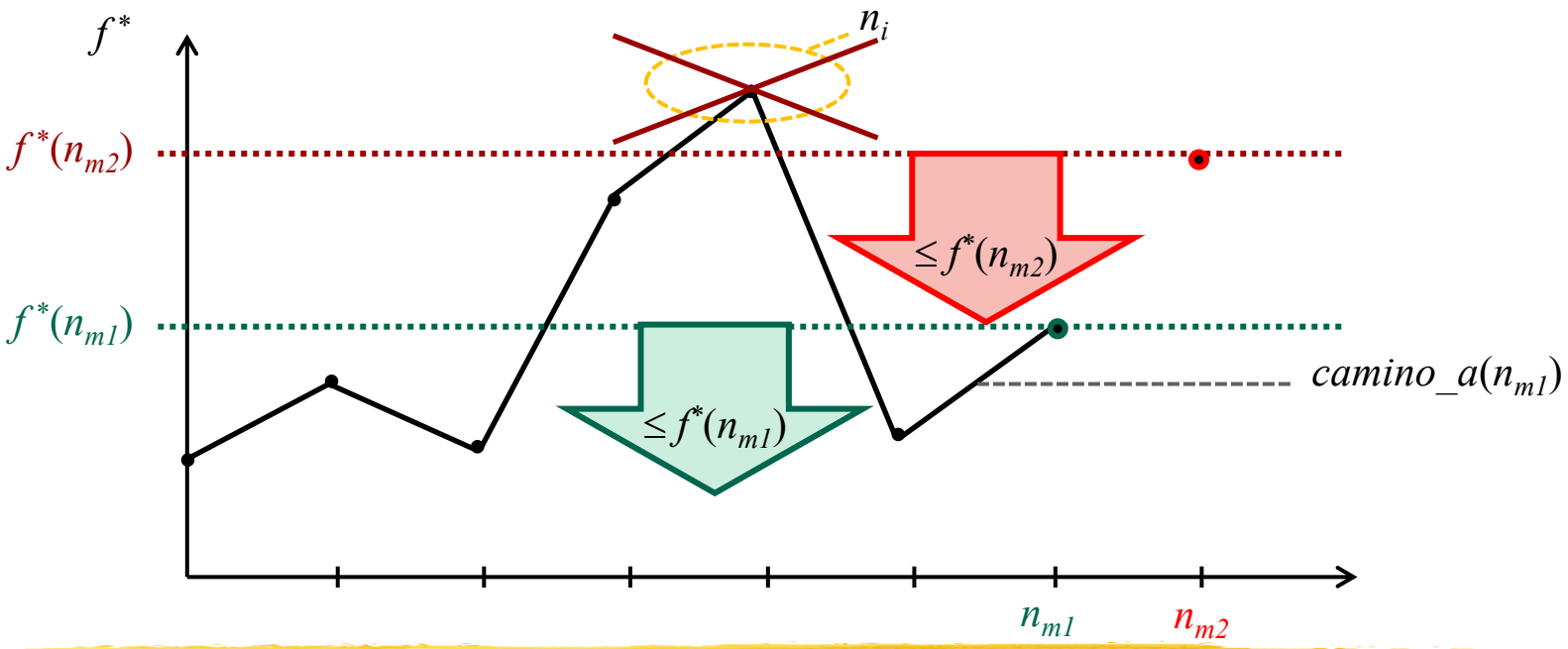


Optimalidad de A*

Visualización de la demostración

Sean n_{m1} y n_{m2} dos nodos meta cualesquiera distintos, y n_{m1} el nodo meta de menor coste (es decir: $g(n_{m1})<g(n_{m2})$).

4. Por tanto, para cualesquiera dos nodos meta, A^* encuentra el de menor coste antes, lo que implica que es óptimo.



Completitud de A*

Teorema 2: El método A^* es *completo*

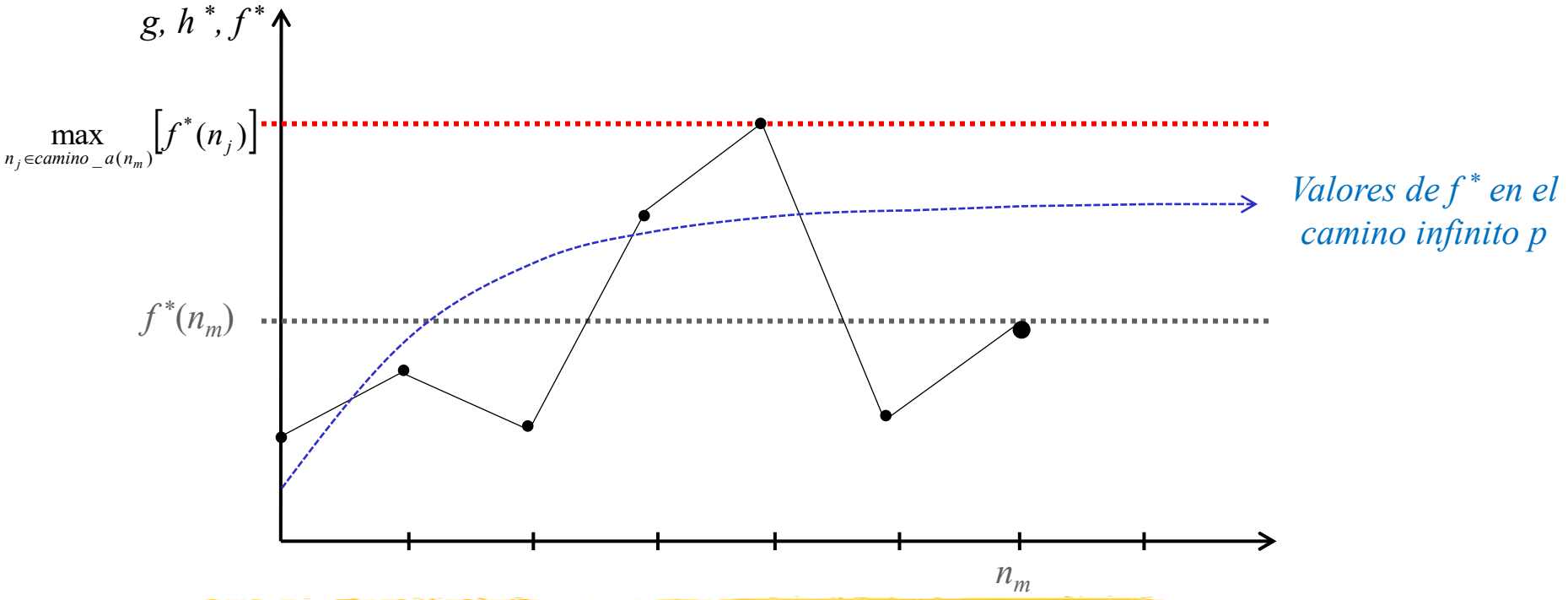
Prueba:

- sea n_m un nodo meta y *camino_a*(n_m) el conjunto de nodos en el camino de la raíz a n_m. Suponga que n_m no es encontrado por el método A^*
  - ya que el número de *sucesores* de un nodo es finito, debe haber un *camino* infinito p, tal que se expanden todos los nodos n_i de p antes de n_m
  - los valores de g a lo largo de p son una secuencia de números naturales que aumenta estrictamente (el coste de los operadores es > 0) y
  - dado que por definición $h^*(n) \geq 0$, existe algún n_k en p con
$$f^*(n_k) = g(n_k) + h^*(n_k) > \max_{n_j \in camino_a(n_m)} \left[f^*(n_j) \right]$$
  - el algoritmo expande los nodos sucesivamente por valores de f^* crecientes, por lo que todos los nodos en el camino a n_m (incluido n_m) son expandidos antes que el nodo n_k
- contradicción; en consecuencia, el método A^* encuentra el nodo meta n_m

Completitud de A^*

Visualización de la demostración

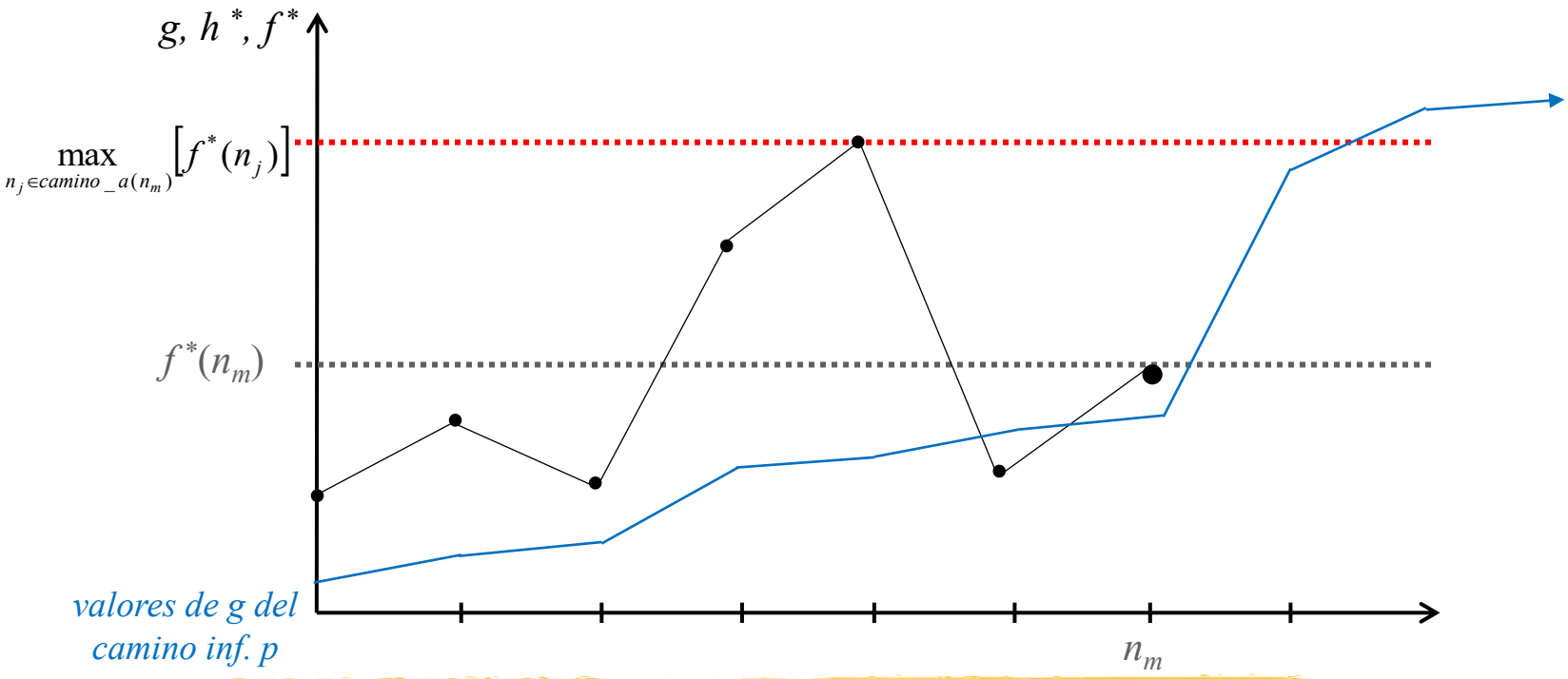
- sea n_m un nodo meta y $camino_a(n_m)$ el conjunto de nodos en el camino de la raíz a n_m. Suponga que n_m no es encontrado por el método A^*
  - ya que el número de *sucesores* de un nodo es finito, debe haber un *camino* infinito p, tal que se expanden todos los nodos n_i de p antes de n_m



Completitud de A^*

Visualización de la demostración

- sea n_m un nodo meta y $camino_a(n_m)$ el conjunto de nodos en el camino de la raíz a n_m. Suponga que n_m no es encontrado por el método A^*
  - los valores de g a lo largo de p son una secuencia de números naturales que aumenta estrictamente (el coste de los operadores es > 0) y

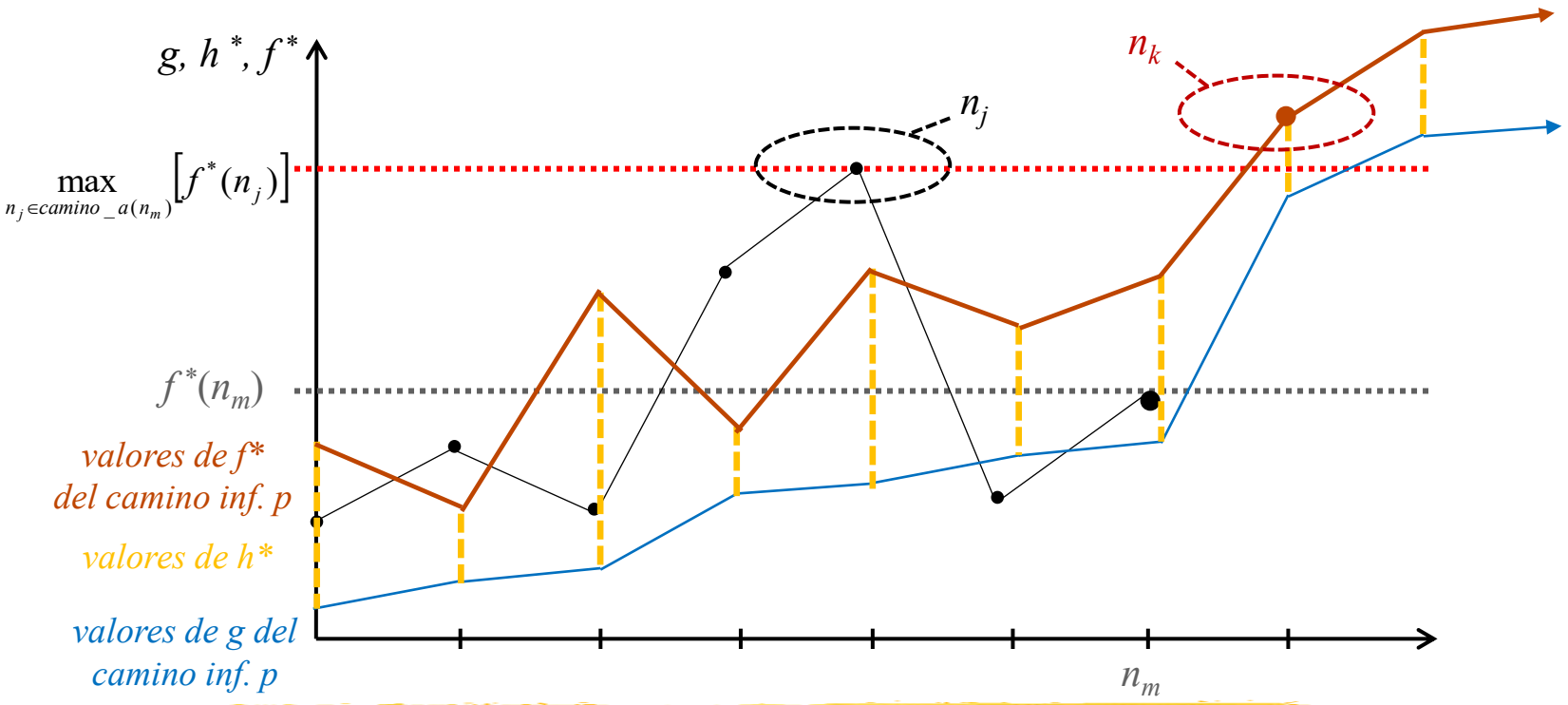


Completitud de A^*

Visualización de la demostración

- sea n_m un nodo meta y $camino_a(n_m)$ el conjunto de nodos en el camino de la raíz a n_m. Suponga que n_m no es encontrado por el método A^*
  - dado que por definición $h^*(n) \geq 0$, existe algún n_k en p con

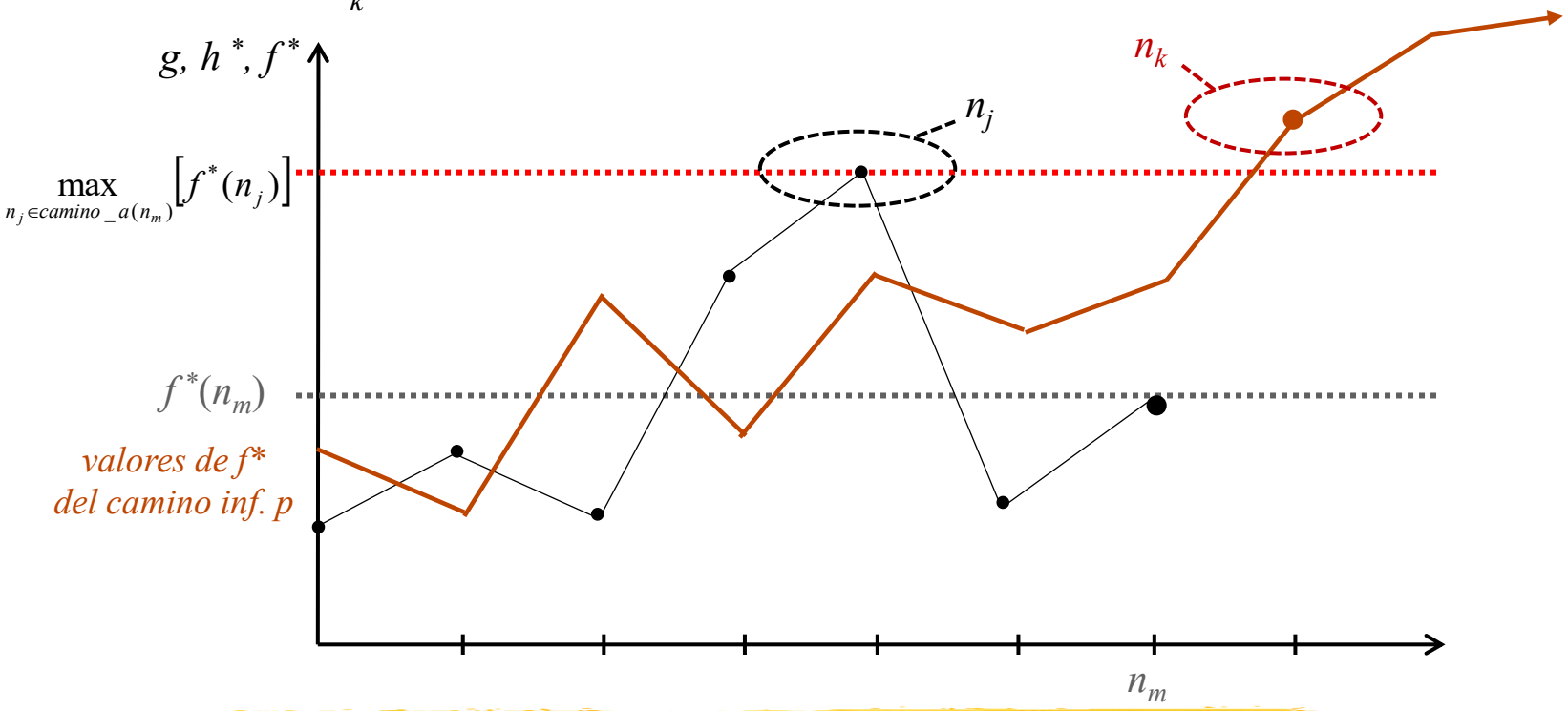
$$f^*(n_k) = g(n_k) + h^*(n_k) > \max_{n_j \in camino_a(n_m)} \left[f^*(n_j) \right]$$



Completitud de A^*

Visualización de la demostración

- sea n_m un nodo meta y $camino_a(n_m)$ el conjunto de nodos en el camino de la raíz a n_m. Suponga que n_m no es encontrado por el método A^*
  - el algoritmo expande los nodos sucesivamente por valores de f^* crecientes, por lo que todos los nodos en el camino a n_m (incluido n_m) son expandidos antes que el nodo n_k



Completitud de A^*

Visualización de la demostración

- contradicción; en consecuencia, el método A^* encuentra el nodo meta n_m

